

Optimizing the DQN Architecture ⚡

In the previous video, we discussed a conceptual architecture where the network takes *both* the State and the Action as input. While correct in theory, it is computationally inefficient.

Most practical implementations (including the one you will use in the practice lab) use a modified architecture.

1. The "Inefficient" Approach (Previous Version)

- **Input:** State vector (s) + Action vector (a) = **12 Inputs**.
- **Output:** Single Q-value ($Q(s, a)$).
- **The Problem:** To decide which action to take in the Lunar Lander (which has 4 actions), you have to run the neural network **4 separate times** (once for each action) to compare the results. This is slow.

2. The "Efficient" Approach (Standard DQN)

Instead of feeding the action as an *input*, we train the network to output Q-values for **all possible actions simultaneously** based solely on the state.

[Image of neural network architecture]

- **Input:** Only the State vector (s) = **8 Inputs**.
- **Hidden Layers:**
 - Layer 1: 64 units.
 - Layer 2: 64 units.
- **Output:** A vector of **4 units**.
 - Unit 1: $Q(s, \text{Nothing})$
 - Unit 2: $Q(s, \text{Left})$
 - Unit 3: $Q(s, \text{Main})$
 - Unit 4: $Q(s, \text{Right})$

Why This is Better

1. **Faster Inference:** When the robot needs to act, we run the network **once**. We get all 4 Q-values immediately and simply pick the maximum.
Best Action = $\text{argmax}([Q_1, Q_2, Q_3, Q_4])$
2. **Faster Training:** When calculating the target Y for the Bellman Equation, we need the term $\max_{a'} Q(s', a')$. With this architecture, we pass the *next state* (s') into the network once, get all potential future Q-values, and simply take the max.